

Presentation Script

How Far Can a PSRAM-less ESP32 Go?

A TinyML Deployment Study for WiFi CSI Human Activity Recognition

Muhammad Ahmad • L1F22BSCS0634 • Section H9 • University of Central Punjab

Target length: about 9–10 minutes • 12 slides

Tips: speak slowly, pause after each key number, and look up at the audience on the bold lines. Advance slides with the right-arrow or space bar. The numbers in this script match the slides exactly.

— Slide 1 • Title

≈25s

On screen: Title, your name, the radiating 3D WiFi scene.

Good morning, everyone. My name is Muhammad Ahmad, and today I will present my research: “How Far Can a PSRAM-less ESP32 Go?” It is a TinyML deployment study for WiFi-based human activity recognition. The question behind the whole project is simple: can the cheapest WiFi chip you can buy, a roughly three-dollar ESP32, run modern activity-recognition AI on its own, with no help from a server?

— Slide 2 • Why WiFi sensing

≈40s

On screen: Three cards: no camera, no wearable, already deployed.

First, why use WiFi for this at all? When a person moves, their body bends and scatters the WiFi signal that is already in the room. A receiver can read those tiny changes and tell what the person is doing, with no camera and nothing worn on the body. That makes it ideal for privacy-sensitive places like homes, hospitals, and elderly care, where a camera would be intrusive or simply not allowed. And because it reuses WiFi that is already everywhere, it can scale very cheaply.

— Slide 3 • What is WiFi CSI HAR

≈45s

On screen: Live CSI spectrogram on the left, four numbered steps on the right.

So what exactly are we sensing? The signal we use is called Channel State Information, or CSI. The idea is this: the router constantly transmits packets; a moving body perturbs them; CSI records how the signal amplitude changes across fifty-two subcarriers over time; and a machine-learning model maps that pattern to an activity such as walking, sitting, or falling. On the left you can see a live CSI spectrogram. That bright band moving through it is, in effect, a body moving through the channel.

— Slide 4 • Live demo (interactive)

≈50s

On screen: Interactive 3D scene. Drag to orbit; click the scenario buttons.

To make this concrete, here is an interactive view. **[Drag to rotate the scene.]** This is the WiFi field radiating from the sensor, and the green figure is a detected person. Let me change scenarios. **[Click “Walking”.]** Here the person is walking, and the floor heatmap follows them. **[Click “Fall”.]** And here a fall is detected, and the system flags it immediately. I want to be honest that this is an illustration of the concept, not my exact classifier output, but it shows what WiFi presence sensing actually looks like.

— Slide 5 • The gap

≈40s

On screen: Prior work vs this work, and the real ESP32 board photo.

Now, here is the gap I found in the literature. Almost everyone who runs these models on a chip uses the more powerful ESP32-S3, which has extra PSRAM memory, or they offload the work to a server. Nobody had characterized the bare classic ESP32: no PSRAM, only about a hundred and fifty kilobytes of usable memory. It is a three-dollar board, and by Espressif’s own ranking it is the

weakest WiFi sensor in the family. Yet that is exactly the chip a real, cost-sensitive product would choose. So my question was: how far can it actually go?

— Slide 6 • Approach: the deployability frontier ≈40s

On screen: Three model tiers and the train to flash pipeline.

My approach was to build what I call a deployability frontier. I took three tiers of models: classical learners like decision trees, tiny neural networks, and five standard deep architectures. I ran all of them across two public datasets, each with seven activities. For every single model I asked one honest question: does it actually convert, fit, and run on the bare chip? And importantly, I tested the CSI-HAR dataset leave-one-user-out, which means the model is always tested on a person it never saw during training.

— Slide 7 • Finding 1: deployment is not the wall ≈45s

On screen: Arena sizes (10 to 54 kB) and the memory chart.

My first finding genuinely surprised me. I expected the deep models to be too big, but they are not. I flashed them to the real board, and a full deep CNN, and even a Transformer, both run on the bare ESP32, using only ten to fifty-four kilobytes. The trick is that the weights sit in the four-megabyte flash, and only the activations need the scarce SRAM. So there is no general memory wall. The chip is far more capable than people assume.

— Slide 8 • The real barriers ≈35s

On screen: Table: CNN and Transformer fit; GRU, SSM, KAN do not.

So what does fail, and why? Two narrow things, and neither is about memory size. First, convertibility: the recurrent and state-space models compile to operators that TensorFlow Lite Micro simply does not support, so they never reach the chip. Second, the Kolmogorov-Arnold network converts but fails to allocate at runtime. Everything else, the CNN, the Transformer, the tiny models, and the classical ones, fits and runs.

— Slide 9 • The generalization gap (headline) ≈50s

On screen: The signal dropping from 96% to 63%, the “-29 points” marker.

And this is the most important slide of the talk. If deployment is not the wall, then what is? It is generalization. Watch what happens to the same model when I change only one thing: how the data is split. On a random split, which lets the model memorize each person, it scores ninety-six percent. But test it on a person it has never seen, and it falls to sixty-three percent. That is a twenty-nine-point collapse, and it happens to every model, regardless of size. A random split basically lets the model cheat by memorizing each user’s body signature. So the real open problem is not fitting the model on the chip; it is recognizing someone new.

— Slide 10 • Results at a glance ≈30s

On screen: Stat cards and the accuracy chart.

To put numbers on it: a tiny CNN reaches about ninety-seven percent on the saturated benchmark, using only eight to thirteen kilobytes of RAM, and running in fourteen to a hundred and seventeen milliseconds on the real board. The classical Random Forest runs in about forty-five microseconds. So the small models are not a compromise; they match the deep models at a tiny fraction of the cost.

— Slide 11 • Takeaways ≈35s

On screen: Four numbered takeaways.

So, four takeaways. One: deployment is solved; even deep models fit the cheapest ESP32. Two: generalization is the real wall, with about a twenty-nine-point drop on a new person. Three: we should report subject-independent accuracy, because random-split numbers are misleading. And four: small models are the smart default for this hardware.

— Slide 12 • Thank you / Q&A

≈15s

On screen: Closing slide with your details.

To sum up: the cheapest WiFi chip can run this AI just fine. The next frontier is helping it recognize a person it has never seen before. Thank you very much for listening. I would be happy to take any questions.

Likely questions and short answers

- **Why is CSI-HAR accuracy low (around 63%)?** Because it is tested on a completely unseen person. That is the honest, real-world number; random splits hide it.
- **Did you test on real WiFi live?** I used two public datasets measured on real hardware. The on-device latency and memory are measured on my physical ESP32.
- **How would you close the generalization gap?** A short per-user calibration, or cross-subject methods like domain adaptation, rather than a bigger model.
- **Why does the KAN fail?** It converts to int8 but the default runtime cannot allocate its quantized polynomial layers; it is a tooling limit, not a fundamental one.